Verteilte Systeme - Virtualisierung und Container

Deployment-Technologien - eine Einführung

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 1.1.0

Folien: [HTML] https://delors.github.io/ds-containers/folien.de.md.html
[PDF] https://delors.github.io/ds-containers/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Einführung in Virtualisierung und Container

Virtualisierung - allg. Definition

> Virtualisierung bezeichnet in der Informatik die **Nachbildung eines Hard- oder Software-Objekts durch ein ähnliches Objekt vom selben Typ mit Hilfe einer Abstraktionsschicht**.
>
> Dadurch lassen sich virtuelle (d. h. nicht-physische) Geräte oder Dienste wie emulierte Hardware, Betriebssysteme, Datenspeicher oder Netzwerkressourcen erzeugen.
>
> Dies erlaubt es etwa, Computer-Ressourcen (insbesondere im Server-Bereich) transparent zusammenzufassen oder aufzuteilen, oder ein Betriebssystem innerhalb eines anderen auszuführen. Dadurch können u. a. mehrere Betriebssysteme auf einem physischen Server oder „Host“ ausgeführt werden.
>
> —Stand 5. Oktober 2025 - Wikipedia

Motivation

Beobachtung

Moderne Softwareentwicklung steht vor verschiedenen Herausforderungen:

- **Heterogene Umgebungen**: Software muss auf verschiedenen Betriebssystemen und Hardware-Konfigurationen laufen.
- **Skalierbarkeit**: (Server-)Anwendungen müssen flexibel skalieren können.
- **Isolation**: Verschiedene (Server-)Anwendungen sollen sich nicht gegenseitig beeinträchtigen.
- **Portabilität**: Code soll einfach zwischen verschiedenen Umgebungen übertragbar sein.
- **Ressourceneffizienz**: Hardware soll optimal genutzt werden.

Was ist Virtualisierung?

Definition

Virtualisierung[1] ermöglicht es, mehrere virtuelle Maschinen (VMs) oder Container auf einer physischen Hardware auszuführen.

Ziel der Virtualisierung

- Abstraktion der Hardware
- Isolation zwischen Anwendungen
- Optimale Ressourcennutzung
- Flexibilität bei der Bereitstellung

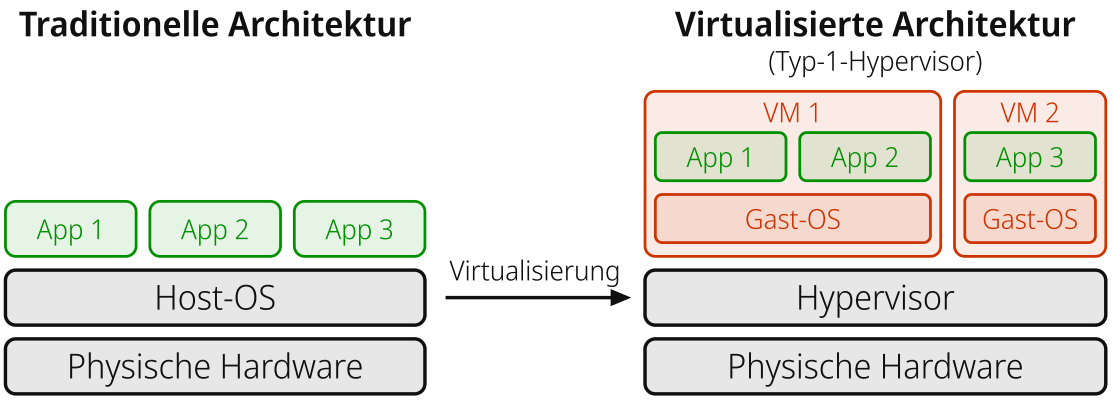
Arten der Virtualisierung

- **Bare-Metal Virtualisierung**: Virtualisierung „direkt“ auf der Hardware
- **Hosted Virtualisierung**: Virtualisierung auf einem Host-Betriebssystem
- **Containervirtualisierung**: Virtualisierung von Systemressourcen eines Betriebssystems; die Container teilen sich jedoch den Kernel des Host-Betriebssystems.
- **Anwendungsvirtualisierung**: Ausführung einer Anwendung in einer exklusiven, teilweise oder vollständig isolierten virtuellen Umgebung.

---

[1] Im Folgenden konzentrieren wir uns auf die Virtualisierung serverseitiger Anwendungen sowie auf Virtualisierung im Kontext der Softwareentwicklung. Virtualisierung von anderen Ressourcen – z. B. Speicher oder Netzwerken – betrachten wir hier nicht.

Architekturunterschiede



Bare-Metal Virtualisierung

Bei der **Bare-Metal Virtualisierung** (auch Type-1 Hypervisor) läuft die Virtualisierungsschicht direkt auf der Hardware, ohne ein Host-Betriebssystem.

Vorteile

★**Bessere Performance**: Kein Overhead durch Host-OS

★**Höhere Sicherheit**: Weniger Angriffsfläche

★**Direkter Hardware-Zugriff**: Optimale Ressourcennutzung

★**Enterprise-fähig**: Für produktive Umgebungen geeignet

Nachteile

✗**Komplexität**: Schwerer zu installieren und zu verwalten

✗**Hardware-Abhängigkeit**: Hardware-Unterstützung erforderlich

✗**Kosten**: ggf. Lizenzkosten für Hypervisor-Software

Beispiele für Bare-Metal Hypervisoren:

- Xen

  XenServer (zeitweise: Citrix Hypervisor)
- KVM (Kernel-based Virtual Machine; Teil des Linux-Kernels)

  Insbesondere *Proxmox VE* (KVM für VMs + LXC für Systemcontainer)[2]
- VMware vSphere/ESXi
- Microsoft Hyper-V (Server)
- ...

---

[2] Proxmox VE ist eine Debian-Distribution; der Linux-Kernel wird über KVM zum Hypervisor. Seit Version 9.1 können LXC-Container auch aus OCI-Images (z. B. von Docker Hub) erzeugt werden.

Hosted Virtualisierung

Bei der **Hosted Virtualisierung** (auch Type-2 Hypervisor) läuft die Virtualisierungsschicht als Anwendung auf einem Host-Betriebssystem.

Vorteile

★**Einfache Installation**: Wie normale Software

★**Flexibilität**: Verschiedene Gast-Betriebssysteme möglich (über Hardwarearchitekturgrenzen hinweg nur mittels Emulation, z. B. QEMU)

★**Entwicklungsumgebung**: Ideal für Tests und Entwicklung

★**Benutzerfreundlich**: Grafische Oberflächen verfügbar

Nachteile

✗**Performance-Overhead**: Host-OS verbraucht Ressourcen

✗**Sicherheitsrisiken**: Host-OS als zusätzliche Angriffsfläche

✗**Begrenzte Skalierbarkeit**: Weniger VMs pro Host möglich

Beispiele für Hosted Hypervisoren:

- QEMU (Open Source - kann Virtualisierung und Emulation)[3]

  (UTM.app erlaubt auf macOS eine einfache Nutzung von QEMU.)
- VMware Workstation
- Oracle VirtualBox
- Parallels Desktop
- ...

---

[3] QEMU kann auch zusammen mit KVM betrieben werden.

2. Container-Technologien

Was sind Container?

Definition: Container

Leichtgewichtige, portable Einheiten, die Anwendungen zusammen mit allen notwendigen Abhängigkeiten (Code, Runtime, Bibliotheken) verpacken.

Sie teilen sich den Kernel des Host-Betriebssystems, bieten aber isolierte Prozess- und Nutzerbereiche, wodurch Anwendungen unabhängig voneinander laufen können.

Technische Grundlagen (Linux)

- **Namespaces**: Isolation von Nutzern, Prozessen, Netzwerk, Dateisystem
- **Control Groups (cgroups)**: Ressourcenbeschränkung (CPU, Speicher, ...)
- **Union File Systems**: Effiziente Speicherung von Layern (keine Isolation, sondern Speicherung)

Technische Grundlagen von Containern unter Linux

Namespaces: Isolieren unterschiedliche Systemressourcen, sodass Prozesse in einem Container „ihre eigene Welt" sehen: Jeder Container hat seine eigenen Benutzer und Gruppen und sieht nur seine eigenen Prozesse. Prozesse aus anderen Containern oder dem Host sind unsichtbar. Container haben eigene Netzwerkschnittstellen, IP-Adressen und Ports und können z. B. auf Port 80 lauschen, ohne Host oder andere Container zu stören. Weiterhin hat jeder Container eine eigene Sicht auf das Dateisystem. Ein Container kann sein Root-Dateisystem haben, ohne den Host oder andere Container zu verändern.

Control Groups (cgroups): Regeln und beschränken die Ressourcennutzung, um fairen Zugriff zu gewährleisten: insbesondere CPU-Zeit, Speicherbenutzung und Block-I/O. (Netzwerkbandbreite wird nicht direkt über cgroups begrenzt, sondern z. B. mit `tc`; cgroups dienen dabei nur der Klassifizierung der Pakete.)

Union File Systems (UnionFS / OverlayFS): Ermöglichen effiziente, schichtweise Speicherung, da zum Beispiel das Basis-Image + Container-spezifische Änderungen als separate Layer gespeichert werden.

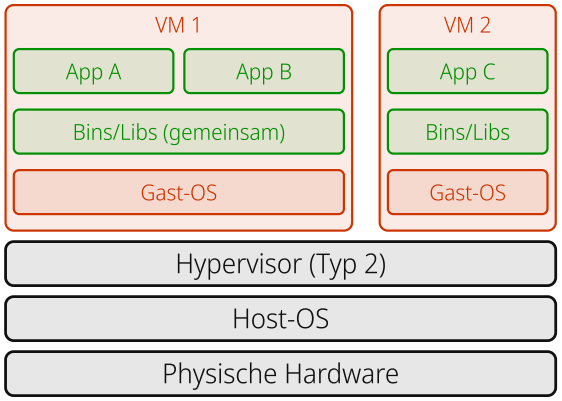
Systemcontainer vs. Anwendungscontainer

Systemcontainer (z. B. LXC, Proxmox VE): Enthalten ein komplettes Linux-Userland inkl. eigenem Init-System und werden ähnlich wie eine (leichtgewichtige) VM betrieben und verwaltet.

Anwendungscontainer (z. B. Docker, Podman): Enthalten nur eine Anwendung mit ihren Abhängigkeiten; typischerweise läuft genau ein (Haupt-)Prozess pro Container.

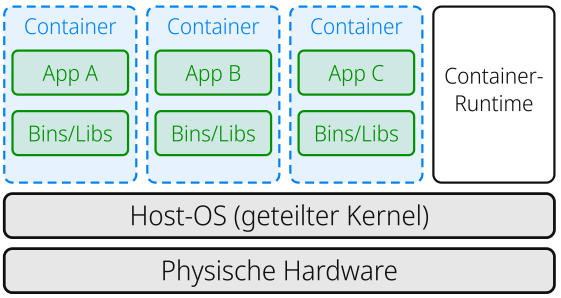
Container vs. virtuelle Maschinen

Virtuelle Maschinen



je VM ein vollständiges Gast-OS (inkl. Kernel)

Container



alle Container teilen den Host-Kernel

Vergleich: Container vs. VMs

Virtuelle Maschinen

Vorteile:

- ★ Vollständige Isolation
- ★ Verschiedene Betriebssysteme möglich
- ★ Hohe Sicherheit

Nachteile:

- ✗ Hoher Ressourcenverbrauch
- ✗ Langsame Startzeiten
- ✗ Komplexe Verwaltung
- ✗ Weniger portabel

Container

Vorteile:

- ★ Schneller Start
- ★ Geringer Ressourcenverbrauch
- ★ Hohe Portabilität
- ★ Einfache Skalierung

Nachteile:

- ✗ Weniger Isolation
- ✗ Gleicher Kernel-Typ erforderlich (Linux-Container benötigen einen Linux-Kernel)
- ✗ Sicherheitsrisiken bei Root-Zugriff

3. Containerisierung mit Docker

Docker - Container-Infrastruktur

Docker ist eine Open-Source-Plattform zur Entwicklung, Bereitstellung und Verwaltung von containerisierten Anwendungen.

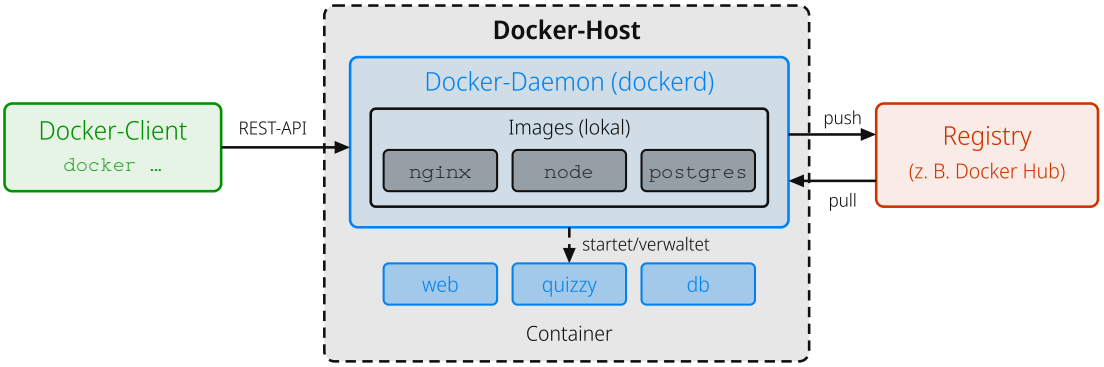
Docker-Komponenten

- **Docker Engine**: Runtime für Container
- **Docker Images**: Vorlagen für Container
- **Docker Containers**: Laufende Instanzen
- **Docker Compose**: Definition und Start von Multi-Container-Anwendungen (auf einem Host)

Docker-Architektur

- **Docker Daemon**: Hintergrundprozess, der Images lädt sowie Container verwaltet und steuert
- **Docker Client**: Kommandozeilen-Tool zum Interagieren mit dem Daemon.
- **Docker Registry**: Dienst, der Image-Repositories speichert und bereitstellt
- **Docker Hub**: Öffentliche Registry

Docker-Architektur



Docker-Images und Container

Ein **Docker-Image** ist eine unveränderliche Vorlage, die alle notwendigen Komponenten für eine Anwendung enthält.

Image-Eigenschaften

- **Layered Architecture**: Images bestehen aus mehreren Layern
- **Immutable**: Images können nicht verändert werden
- **Portable**: Funktionieren auf verschiedenen Systemen
- **Versioned**: Verschiedene Versionen eines Images möglich

Container-Lifecycle

- **Created**: Container aus Image erstellt (`docker create`)
- **Running**: Container läuft (`docker start`; `docker run` = create + start)
- **Paused**: Prozesse angehalten (`docker pause`)
- **Exited**: Container gestoppt (`docker stop`)
- **Removed**: Container gelöscht (`docker rm`)

Docker-Befehle

Image-Verwaltung:

- `docker pull <image>` - Image herunterladen
- `docker images` - Alle Images anzeigen
- `docker build -t <name> .` - Image erstellen
- `docker rmi <image>` - Image löschen

Container-Verwaltung:

- `docker run <image>` - Container erzeugen und starten
- `docker ps` - Laufende Container anzeigen
- `docker stop <container>` - Container stoppen
- `docker rm <container>` - Container löschen

Informationen:

- `docker logs <container>` - Container-Logs anzeigen
- `docker logs -f <container>` - Container-Logs fortlaufend verfolgen
- `docker exec -it <container> /bin/sh` - Shell im Container öffnen

4. Container-Orchestrierung

Was ist Container-Orchestrierung?

Container-Orchestrierung ist die Automatisierung der Bereitstellung, Verwaltung, Skalierung und Vernetzung von Container-Anwendungen.

Gegenstand der Orchestrierung:

- **Deployment**: Automatische Bereitstellung von Containern
- **Scaling**: Dynamische Skalierung basierend auf Last
- **Load Balancing**: Verteilung der Anfragen
- **Service Discovery**: Auffinden von Services
- **Health Monitoring**: Überwachung der Container-Gesundheit
- **Rolling Updates**: Updates ohne Downtime

Container-Orchestrierungs-Tools

Kubernetes

- **Open Source**, ursprünglich von Google, heute Cloud Native Computing Foundation (CNCF)
- **De-facto-Standard** für Container-Orchestrierung
- **Umfangreiche Features**
- **Große Community**
- **Komplexe Einrichtung**

Docker Swarm

- **Native Docker-Lösung**
- **Einfache Einrichtung**
- **Weniger Features** als Kubernetes
- **Gut für kleinere Projekte**
- **Einfache Verwaltung**
- Nur noch Bugfixes

HashiCorp Nomad

- **Ein einzelnes Binary**
- **Nicht nur Container** (auch VMs, Java-Anwendungen, …)
- **Hochskalierbar**
- **Einfacher als Kubernetes**
- **Weniger verbreitet**

Exemplarische Verwendung von Docker

Docker-basiertes Deployment einer Web-Anwendung

Entwicklung und Deployment einer einfachen Web-Anwendung mit Docker.

1. **Docker-Installation prüfen**
2. **Einfache Web-Anwendung erstellen**
3. **Dockerfile erstellen**
4. **Docker-Image bauen**
5. **Container starten und testen**
6. **Docker Compose für Multi-Container-Setup** (optional)
7. **Multi-Container-Anwendung starten** (optional)

Schritt 1: Docker-Installation prüfen

```
# Docker-Version prüfen
docker --version

# Docker-Status prüfen
docker info

# Ersten Container testen
docker run hello-world
```

Schritt 2: Web-Anwendung erstellen (`index.html`)

```
<!DOCTYPE html>
<html lang="de">
<head>
    <meta charset="utf-8">
    <title>Docker Demo App</title>
</head>
<body>
    <h2>Willkommen zur Docker-Demo!</h2>
    <p>Diese Anwendung läuft in einem Docker-Container.</p>
    <p>Aktuelle Zeit: <span id="time"></span></p>
    <script>
        document.getElementById('time').textContent =
            new Date().toLocaleString();
    </script>
</body>
</html>
```

Schritt 3: `Dockerfile` erstellen

```
# Basis-Image verwenden (Alpine-Linux mit einem vorinstallierten nginx)
FROM nginx:alpine

# Arbeitsverzeichnis innerhalb des Containers setzen
WORKDIR /usr/share/nginx/html

# HTML-Datei kopieren (kopiert lokale Datei in den Container relativ zum WORKDIR)
COPY index.html .

# Port 80 freigeben
EXPOSE 80
```

```
12
13 # Nginx im Vordergrund starten (da sich der Container sonst sofort beendet)
14 CMD ["nginx", "-g", "daemon off;"]
```

Schritt 4: Docker-Image bauen

```
1 # Image bauen mit dem Tag "webapp-demo"
2 docker build -t webapp-demo .
3
4 # Images anzeigen
5 docker images
6
7 # Image-Details anzeigen
8 docker inspect webapp-demo
```

Schritt 5: Container starten und testen

```
1 # Container mit dem Namen "webapp-container" im Hintergrund (-d) starten
2 docker run -d -p 8080:80 --name webapp-container webapp-demo
3
4 # Container-Status prüfen
5 docker ps
6
7 # In Browser testen: http://localhost:8080
8
9 # Container-Logs anzeigen
10 docker logs webapp-container
11
12 # Container stoppen
13 docker stop webapp-container
14
15 # Container (nicht Container-Image) löschen
16 docker rm webapp-container
```

Schritt 6: Docker Compose für Multi-Container (`docker-compose.yml`)

```
1  # Nur zur Illustration: Die statische Web-Anwendung nutzt die DB nicht.
2  services:
3    web:
4      build: .
5      ports: ["8080:80"]
6      depends_on: [db]
7      environment: [DB_HOST=db]
8
9    db:
10     image: postgres:17
11     environment:
12       POSTGRES_DB: demo
13       POSTGRES_USER: demo
14       # Nur für die Demo! Echte Passwörter gehören in Secrets.
15       POSTGRES_PASSWORD: demo123
16     volumes: [postgres_data:/var/lib/postgresql/data] # persistent
17
18 volumes:
19   postgres_data:
```

Schritt 7: Anwendung (bestehend aus mehreren Containern) starten

```
1 # Multi-Container-Setup starten
2 docker compose up -d
3
```

```
# Status prüfen
docker compose ps

# Logs anzeigen
docker compose logs

# Setup stoppen
docker compose down
```

5. Zusammenfassung

Konzepte

- **Virtualisierung** ermöglicht effiziente Nutzung von Hardware-Ressourcen
- **Container** bieten eine leichtgewichtige Alternative zu VMs
- **Docker** ist die verbreitetste Container-Plattform (Standard ist das OCI-Format)
- **Orchestrierung** automatisiert das Management von Container-Anwendungen
- **Multi-Container-Setups** ermöglichen komplexe Anwendungsarchitekturen

Best Practices für Docker

Image-Erstellung

- **Non-Root User** für Sicherheit
- **Layer-Caching** optimieren
- **Multi-Stage Builds** und `.dockerignore` nutzen
- **Feste Image-Tags** statt `latest` verwenden

Container-Management

- **Health Checks** implementieren
- **Resource Limits** setzen
- **Logging** konfigurieren
- **Backup-Strategien** für Volumes

Sicherheit

- **Minimale Images** verwenden
- **Secrets** nicht im Image speichern
- **Netzwerke** der Container gezielt trennen
- **Regelmäßige Aktualisierungen** durchführen

Ausblick

Die Container-Technologie entwickelt sich weiter:

- **Serverless Container** (AWS Fargate, Azure Container Instances)
- **WebAssembly (WASM)** als Alternative zu Containern
- **Edge Computing** mit Container-Technologie
- **GitOps** für automatisiertes Deployment
- **Service Mesh** für Microservice-Kommunikation

∴ Schlussfolgerung

Container und Virtualisierung sind fundamentale Technologien für moderne Softwareentwicklung und -deployment. Die praktische Anwendung mit Docker bietet eine solide Grundlage für weiterführende Themen wie Kubernetes und Cloud-Native-Entwicklung.

✎ Übung: Erste Schritte

5.1. Webserver im Docker Container

Loggen Sie sich auf dem Server ein und versuchen Sie alle Schritte nachzuvollziehen, die notwendig sind, um ein Docker-Image mit Nginx zu bauen und danach zu starten. Orientieren Sie sich an dem Beispiel aus den Vorlesungsfolien, aber nutzen Sie eine eigene `index.html`.

> ▲ Achtung!
>
> Spezifizieren Sie beim Start des Containers ein Portmapping (z. B. `8100:80`) passend zu den Ihnen zugeteilten Ports!

Stellen Sie sicher, dass Ihr Server läuft, indem Sie die Webseite aufrufen.

Verfolgen Sie das Log (`docker logs -f`), um die Zugriffe auf Ihre Webseite zu sehen.

✎ Übung: Node.js im Container

5.2. Node.js Server im Container laufen lassen

1. Kopieren Sie die Ressourcen (player.html, admin.html, game.js, package.json) auf Ihren Server in ein neu angelegtes Verzeichnis
2. Ihr Docker-Image soll node.js und alle Ressourcen enthalten.

   D. h. Ihr `Dockerfile` muss eine Referenz auf ein entsprechendes Image mit node.js enthalten und die Ressourcen in das Image kopieren.

3. Passen Sie das `Dockerfile` so an, dass beim Bauen des Docker-Images die benötigten Bibliotheken mit installiert werden. Sie müssen dafür den Befehl `npm install --omit=dev` während des Bauens mit Hilfe von `RUN` im `Dockerfile` ausführen.
4. Passen Sie den Kommandozeilenbefehl (`CMD`) im `Dockerfile` so an, dass der Node Server passend gestartet wird.

   ▲ Achtung!

   Die Anwendung läuft standardmäßig auf Port 8800 (änderbar über die Umgebungsvariable `QUIZZY_PORT`). Verwenden Sie beim Portmapping einen der Ihnen zugeteilten Ports.